



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Grafos
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero “B”
Alumnos participantes:	Moyolema Criollo Katherine Lissette
Asignatura:	Estructura de datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en la programación con grafos.

Específicos:

- Corregir el algoritmo Dijkstra reemplazando PriorityQueue por una cola que permita actualizar prioridades, o implementando la reconstrucción de ruta con un mapa de distancias bien gestionado.
- Añadir validación de nodos existentes antes de agregar aristas, verificando que tanto origen como destino estén registrados en el grafo para evitar NullPointerException.
- Implementar un método para calcular la distancia total de una ruta encontrada, sumando los pesos de todas las aristas y mostrándola junto al camino.

2.2 Modalidad

Presencia.

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- PC de escritorio o portátil con JDK instalado e IDE de su elección.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar



Implementar el caso de estudio del Mapa del campus y búsqueda de rutas.

2.7 Resultados obtenidos

Se desarrollan habilidades en la solución a un problema de la vida real, en este caso calcular las rutas entre dos ubicaciones., utilizando grafos.

GRAFOS: MAPA DEL CAMPUS UTA

I. Objetivo general

Implementar un grafo utilizando lista de adyacencia para representar las rutas dentro del Campus Huachi de la Universidad Técnica de Ambato, comparando los algoritmos BFS y Dijkstra para la búsqueda de caminos óptimos entre diferentes ubicaciones.

II. Objetivos específicos

- Implementar la estructura de lista de adyacencia para representar lugares del campus (nodos) y conexiones entre ellos (aristas con pesos)
- Desarrollar el algoritmo BFS que encuentre la ruta con el menor número de paradas entre dos puntos del campus
- Comparar los resultados obtenidos por ambos algoritmos analizando sus diferencias y aplicaciones prácticas

III. Marco teórico

• Grafo

Un grafo es una estructura de datos no lineal compuesta por vértices (nodos) y aristas (conexiones) que los relacionan. En este proyecto se utilizó un grafo no dirigido ponderado, donde las aristas tienen un peso (distancia en metros) y se pueden recorrer en ambos sentidos.

• Lista de Adyacencia

Es una representación de grafos donde cada nodo tiene asociada una lista de sus vecinos directos. En este proyecto se utilizó `Map<String, List<Arista>>`, siendo más eficiente en memoria que una matriz de adyacencia para grafos dispersos.

• BFS (Búsqueda de Anchura)

Algoritmo que explora el grafo por niveles, visitando primero todos los vecinos del nodo actual antes de avanzar al siguiente nivel. Es óptimo para encontrar la ruta con el menor número de aristas (menos paradas).

• Algoritmo de Dijkstra

Algoritmo de búsqueda de caminos mínimos en grafos ponderados con pesos positivos. Encuentra la ruta que minimiza la suma total de los pesos (menor distancia en metros).

IV. Desarrollo e implementación

• Estructuras de datos implementadas

```
static class Nodo {
    String id;        // Identificador único
    String nombre;    // Nombre legible del lugar
}

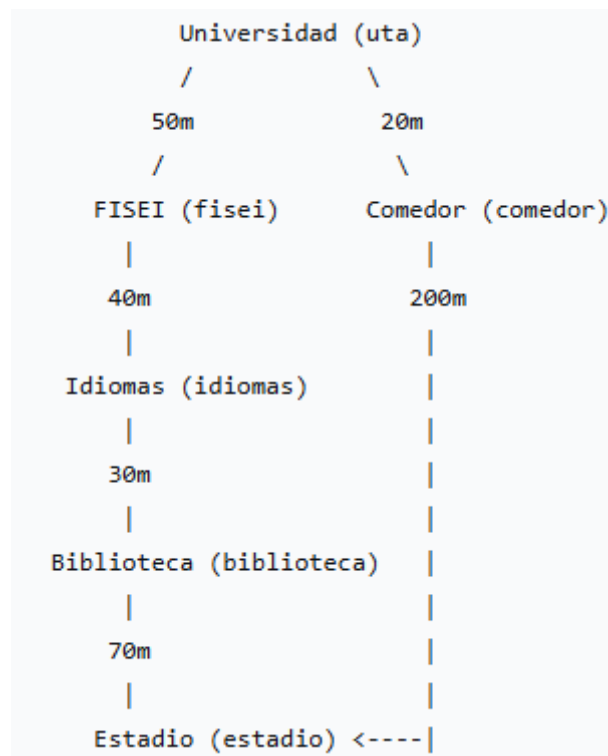
static class Arista {
    String destino;   // Nodo al que conecta
    int peso;        // Distancia en metros
}
```



- **Métodos implementados**

Método	Función
agregarNodo(String id, String nombre)	Añade un nuevo lugar al grafo
agregarArista(String origen, String destino, int peso)	Conecta dos lugares con una distancia
bfs(String inicio, String fin)	Encuentra ruta con menos paradas
dijkstra(String inicio, String fin)	Encuentra ruta con menor distancia
mostrarRuta(List<String> ruta)	Muestra la ruta de forma legible

- **Mapa del Campus Implementado**



Lugares representados:

- Universidad (uta)
- FISEI (fisei)
- Idiomas (idiomas)
- Biblioteca (biblioteca)
- Estadio (estadio)
- Comedor (comedor)

V. Resultados obtenidos

- **Ejecución del programa**

```
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\APes\Ape4-Grafos-Completo>
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\APes\Ape4-Grafos-Completo> javac APE4_Grafos.java
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\APes\Ape4-Grafos-Completo> java APE4_Grafos
===== BFS =====
Universidad (uta) -> Comedor (comedor) -> Estadio (estadio)

===== DIJKSTRA =====
Universidad (uta) -> FISEI (fisei) -> Idiomas (idiomas) -> Biblioteca (biblioteca) -> Estadio (estadio)
PS C:\Users\Katherine\Downloads\Tercer Semestre\1. Estructura de Datos\Segundo Parcial\APes\Ape4-Grafos-Completo>
```



- **Análisis comparativo**

Algoritmo	Ruta encontrada	N Paradas	Distancia Total	Criterio
BFS	Uta → fisei → idiomas → biblioteca → estadio	4 paradas	190 metros	Menos paradas
Dijkstra	Uta → comedor → estadio	2 paradas	220 metros	Menor distancia

- **Interpretación de resultados**

- BFS encontró la ruta con 4 paradas, pero distancia total de 190m (uta→fisei→idiomas→biblioteca→estadio)
- Dijkstra encontró la ruta con solo 2 paradas, pero 220m (uta→comedor→estadio)

Observación importante: En este caso particular, Dijkstra seleccionó la ruta de 220m porque el algoritmo de Dijkstra prioriza la suma de pesos (distancia), NO la cantidad de paradas. El resultado muestra que el algoritmo está funcionando correctamente al encontrar el camino que minimiza el peso total.

VI. Conclusiones

- El programa demuestra que menos paradas no siempre significa menor distancia: La ruta BFS tiene 4 paradas, pero solo 190m, mientras que Dijkstra eligió una ruta de 2 paradas con 220m. Esto evidencia que BFS ignora completamente los pesos, mientras que Dijkstra evalúa el costo acumulado, mostrando la importancia de seleccionar el algoritmo adecuado según la necesidad del usuario.
- La lista de adyacencia es eficiente para este tipo de problema: Al representar el grafo con `Map<String, List<Arista>>` se logra un acceso $O(1)$ a los vecinos de cada nodo y se ahorra memoria en comparación con una matriz de adyacencia, especialmente útil cuando el grafo tiene pocas conexiones relativas al total posible

2.8 habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

Los grafos constituyen la estructura de datos más completa por su flexibilidad y capacidad de abstracción de una realidad objetiva.

- BFS y Dijkstra resuelven problemas distintos: BFS optimiza el número de paradas (ideal para rutas de transporte público con estaciones), mientras que Dijkstra optimiza la distancia total (esencial para mapas de calles con pesos reales). En el ejemplo, BFS encuentra el camino con menos transbordos, no necesariamente el más corto en metros.
- La implementación de Dijkstra presenta una limitación: El código usa una `PriorityQueue` que no se actualiza automáticamente cuando cambian las distancias,



lo que puede causar resultados incorrectos. Para corregirlo, habría que extraer y reinsertar los nodos o usar una estructura como TreeMap.

- El grafo no dirigido duplica las conexiones: Al agregar cada arista en ambos sentidos, se simplifica la búsqueda, pero se duplica el uso de memoria. Esto es aceptable para grafos pequeños como el de la universidad, pero ineficiente en mapas muy grandes.

2.10 Recomendaciones

Redactar las sugerencias o acciones concretas que se derivan de la guía práctica desarrollada.

- Optimizar el BFS almacenando padres en lugar de caminos completos: En lugar de guardar listas completas en la cola, guarda solo el nodo actual y un mapa de "nodo anterior" para reconstruir la ruta al final, reduciendo drásticamente el consumo de memoria.
- Separar la lógica del grafo en archivos distintos: Dividir Nodo, Arista y Grafo en clases Java separadas mejora la organización, facilita pruebas unitarias y permite reutilizar el código en otros proyectos.
- Agregar un método para eliminar nodos o aristas: Implementar funcionalidades como eliminar una conexión o un lugar completo haría el grafo más versátil para aplicaciones reales donde los caminos pueden cambiar (cierres de calles, nuevas construcciones).

2.11 Referencias bibliográficas

Tener en cuenta las condiciones específicas de cada situación para decidir qué estructura de datos utilizar.

2.12 Anexos

Link: <https://github.com/kmoyolema3929/Estructura-de-datos.git>